HafenCity Universität Hamburg

M.Sc. in Geodesy and Geoinformatics
with specialization in Hydrography

ULi Motion Compensation

Python Scripts Documentation

Author:
Ahsan Ahammed Ansari

Associated Master's Thesis:
Development and Analysis of Motion Compensation Algorithms for the Underwater LiDAR ULi

GitHub Repository Link

Contents

1. Overview

This document provides the documentation for Python scripts developed as part of a master's thesis titled "Development and Analysis of Motion Compensation Algorithms for the Underwater LiDAR ULi" at the HafenCity Universität Hamburg. The scripts were developed for the motion compensation and boresight calibration of point clouds acquired with the ULi system, an underwater laser scanning system developed by Fraunhofer IPM.

1.1 Research Context

The ULi system was mounted on a linear motion unit in a recirculating flume at the Bundesanstalt für Wasserbau (BAW), Hamburg, Germany. Data was acquired while the ULi system was moved linearly, which caused motion-induced distortions in the resulting point clouds. To correct these distortions, trajectory data was collected using a Trimble SPS930 total station which actively tracked a prism mounted on the motion unit.

The ULi system captures points in the TAI time standard, and the total station in UTC time standard. A conversion from UTC to TAI is thus required to ensure consistency between the point cloud and trajectory data.

Two independent motion compensation workflows were developed:

1. **Python Workflow** — A fully reproducible workflow developed in Python operating on `.las` files using rigid body transformations.
2. **Pulsalyzer Workflow** — Using the proprietary *Pulsalyzer* software of the ULi system, adapted to use external trajectory data instead of the onboard INS.

The rigid body transformation applied follows the equation:

$$P_{\text{world}} = R \cdot P_{\text{local}} + T(t) \tag{1.1}$$

where P_{world} is the point coordinate in the total station (world) frame, P_{local} is the point coordinate in the scanner (local) frame from the `.las` file, R is a fixed rotation (boresight) matrix defining the relative orientation between the scanner and the total station, and $T(t)$ is the time-interpolated translation vector from the trajectory.

2. Required Dependencies

The scripts were developed using **Python 3.12.10**. The following external Python libraries are also required in addition to the standard library. It is recommended to use a virtual environment to avoid conflicts.

Library	Version	Purpose
laspy	2.6.1	Reading and writing `.las` point cloud files
numpy	2.3.2	Array operations, linear algebra, numerical computation
open3d	0.19.0	3D point cloud visualization
pandas	2.3.1	Trajectory data manipulation and analysis
scipy	1.16.3	Interpolation, rotation matrix generation
tkinter	-	GUI for boresight calibration tool (Python Standard Library)

Table 2.1: Required Python Libraries

All external libraries can be installed at once using the provided `requirements.txt` file:

```
pip install -r requirements.txt
```

Alternatively, they can be installed individually:

```
pip install laspy==2.6.1 numpy==2.3.2 open3d==0.19.0 pandas==2.3.1 scipy
   ==1.16.3
```

Note: `tkinter` is part of the Python Standard Library and does not require a separate `pip` installation. However, on Linux it may need to be installed manually (e.g., `sudo apt install python3-tk`). To ensure full compatibility, use **Python 3.12.10**, as the behaviour of standard libraries may vary across Python versions.

3. A_json_commander.py

This is a script provided by Fraunhofer IPM, which can be used to send commands using the JSON interface to programs such as Pulsalyzer. This script thus enables a user to control programs (such as Pulsalyzer) programmatically.

3.1 How It Works

The script connects to a local TCP socket on 127.0.0.1:44444 (JSON_Router) and routes JSON-formatted commands to programs and their endpoints.

For more details regarding the JSON interface, its features and detailed usage, visit the local URL `http://localhost:5000/ipm/help/jsoniface_help` after launching **JSONServer**, which is provided by Fraunhofer IPM.

3.2 Key Function

```
send_commands(cmds, routing, host="127.0.0.1", port=44444)
```

Parameter	Type	Description
cmds	list	List of command dictionaries (e.g., {"cmd": "reprocess_pointcloud", ...})
routing	list	Routing path to endpoint (see chapter 8 for format)
host	str	IP address of the host (default: `"127.0.0.1"` (localhost))
port	int	TCP port of the JSON_Router (default: 44444)

3.3 Usage

This script is not run directly. It is to be imported by other scripts, thus controlling programs programmatically. [See chapter 8 for an example]

```
import A_json_commander as json_commander
json_commander.send_commands(cmds, routing)
```

Note: '`A_json_commander.py`' must be placed in the same directory as any script that imports it.

4. B_trajectory_conversion.py

This script converts the raw trajectory file obtained from the total station measurements into the format which is compatible by Pulsalyzer software. This converted trajectory is also used as the trajectory input for the Python workflow explained in chapter 6.

4.1 File Structure

The measurements captured by the total station is logged by the Allterra TS Logger software in a .txt file format. Each row in this file contains a time-of-day timestamp (HHMMSS.SSS format in UTC), previously set station coordinates, a horizontal angle (azimuth), a vertical angle (zenith), and a slope distance.

This script reads the raw trajectory file and performs the following operations:

1. Recomputes Cartesian (X, Y, Z) coordinates of tracked points from the polar coordinates (azimuth, zenith, slope distance) with respect to the total station as origin.
2. Converts from left-handed to right-handed coordinate system (negates Y).
3. Applies a lever arm correction along Z to account for the lever arm offset between the tracked prism and the ULi system.
4. Converts time-of-day timestamps to seconds since UNIX epoch (1 January 1970 00:00:00 UTC) and adjusts for the UTC-to-TAI time standard difference.
5. Outputs the trajectory data in tab-delimited Pulsalyzer format.

4.2 User-Configurable Parameters

Variable	Default	Description
Year, Month, Day	'2025', '08', '06'	Date of data collection
x_rot, y_rot, z_rot	0, 0, 0	Scanner orientation (in degrees)
utc2tai_offset	37	UTC-to-TAI offset in seconds
prism_to_scanner_lever_arm	1.65	Vertical distance (m) from prism to ULi lens
input_trajectory_file	R"..."	Full path to raw .txt trajectory file (e.g.: `R"C:\Folder \file .txt"`)

4.3 Output File

The output is a tab-delimited `.txt` file containing the processed trajectory with the following columns and a corresponding header:

Column	Unit	Description
`time`	seconds (s)	Timestamps since UNIX epoch (including UTC-to-TAI offset)
`x`	metres (m)	Cartesian X coordinate in the right-handed total station frame
`y`	metres (m)	Cartesian Y coordinate in the right-handed total station frame
`z`	metres (m)	Cartesian Z coordinate in the right-handed total station frame with lever arm correction
`roll`	degrees (°)	Scanner roll orientation (Default 0.00°)
`pitch`	degrees (°)	Scanner pitch orientation (Default 0.00°)
`yaw`	degrees (°)	Scanner yaw orientation (Default 0.00°)

The output file is saved in the same directory as the input trajectory file with the following naming convention:

`converted_{Year}_{Month}_{Day}_offset_{utc2tai_offset}_traj_{x_rot}_{y_rot}_{z_rot}.txt`

where,

- `Year, Month, Day` are the date of data collection
- `utc2tai_offset` is the UTC-to-TAI offset in seconds
- `x_rot, y_rot, z_rot` are the scanner orientation offsets

4.4 Example of conversion

Raw trajectory

```
101533.231 548775.866 5937527.313 10.950 208.58550 99.53554 35.764
101533.281 548775.866 5937527.312 10.950 208.58530 99.53544 35.763
101533.331 548775.866 5937527.313 10.950 208.58550 99.53534 35.764
101533.382 548775.867 5937527.313 10.950 208.58570 99.53534 35.764
101533.433 548775.866 5937527.313 10.950 208.58560 99.53534 35.764
```

Converted trajectory

```
time  x y z roll  pitch yaw
1754475370.231000 -35.438322  4.808424  -1.389078 0.000 0.000 0.000
1754475370.281000 -35.437346  4.808178  -1.389029 0.000 0.000 0.000
1754475370.331000 -35.438321  4.808424  -1.388966 0.000 0.000 0.000
1754475370.382000 -35.438306  4.808535  -1.388966 0.000 0.000 0.000
1754475370.433000 -35.438314  4.808479  -1.388966 0.000 0.000 0.000
```

5. C_py_files_gen.py

This script downsamples a `.las` point cloud to 10% and 1% of the original dataset. The downsampled point clouds are saved as both `.las` and `.csv` files. The `.csv` files are used as input for the boresight calibration GUI (chapter 7)

5.1 How It Works

The `.las` file is read using the `laspy` Python library. Downsampling is done by retaining every n^{th} point from the input file - every 10^{th} point for 10% downsampling, and every 100^{th} point for 1% downsampling. The retained points along with all their associated attributes are written to a new `.las` file using `laspy`.

A `.csv` file is then generated from each downsampled `.las` file using `numpy`. The `.csv` files contain only the fields `gps_time, x, y, z`.

5.2 User-Configurable Parameters

Variable	Default	Description
las_file	R"..."	Full path to the input .las file (e.g.: `R"C:\Folder \file .las"`)

5.3 Outputs

All outputs are saved to the same directory as the input `.las` file.

- `10%_<filename>.las`: 10% downsampled point cloud
- `1%_<filename>.las`: 1% downsampled point cloud
- `10%_<filename>.csv`: CSV with columns: `gps_time, x, y, z`
- `1%_<filename>.csv`: CSV with columns: `gps_time, x, y, z`

6. D_python_workflow.py

This is the script for the Python motion compensation workflow. The motion compensation is done by applying a rigid body transformation (rotation + time-interpolated translation) to all points in an input `.las` file.

6.1 How It Works

This script corrects the motion-induced distortions by:

1. Loading the distorted `.las` file.
2. Loading the converted Pulsalyzer format trajectory file and removing duplicate timestamps for linear interpolation.
3. Verifying that the `.las` file timestamps are within the input trajectory time range, if not, stop execution.
4. The trajectory is filtered to the corresponding time window of the input `.las` file.
5. Checking trajectory linearity with the condition $R^2 \geq 0.999$. If condition fails, stop execution.
6. Linear interpolation functions are then built for x, y, and z as a function of time.
7. Building a rotation matrix from user-defined boresight Euler angles (intrinsic XYZ).
8. Processing the `.las` file in chunks to manage memory.
9. Rotating each point in a chunk using the rotation matrix and adding the linearly interpolated position from the linear interpolation functions.
10. Each chunk is written incrementally to an output `.las` file.

6.2 User-Configurable Parameters

Variable	Default	Description
input_las_file	R"..."	Path to distorted .las file (e.g.: `R"C:\Folder \file .las"`)
input_pulsalyzer_trajectory_file	R"..."	Path to converted trajectory .txt file (e.g.: `R"C:\Folder \file .txt"`)
x_rot, y_rot, z_rot	0	Boresight rotation angles in degrees
chunk_size	1000000	Points processed per iteration

6.3 Rotation Convention

The rotation matrix is built using the **Intrinsic XYZ** rotation convention.

The module `scipy.spatial.transform` of the Python library `scipy` is used for building the rotation matrix. The method used for building the rotation matrix from input angles in degrees following the intrinsic XYZ rotation convention is:

```
Rotation.from\_euler('XYZ', [x\_rot, y\_rot, z\_rot], degrees=True).as\
    _matrix()
```

For more information on the usage of the module and rotation convention notations, refer to the documentation available at:

https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.spatial.transform.Rotation.html

The boresight angles obtained in the thesis were:

$$x\_rot = 2.5^\circ, \quad y\_rot = 184^\circ, \quad z\_rot = 48^\circ$$

7. E_boresight_calibration.py

This script is an interactive GUI application for visually determining the boresight rotation angles. It uses Open3D for 3D visualization and Tkinter for the control panel. A binary (.exe) file of this script is available under releases in the GitHub repository.

7.1 How It Works

The boresight calibration process involves adjusting three rotation angles (Roll/X, Pitch/Y, Yaw/Z) until the point cloud appears geometrically correct. The tool applies the rigid body transformation in real time as sliders are moved. The angles can also be adjusted as text inputs near the sliders in the control panel.

7.2 Input File Requirements

File	Format	Columns Required
Point Cloud	.csv	`gps_time, x, y, z`
Trajectory	.txt	`time, x, y, z, roll, pitch, yaw`

7.3 Usage

This application opens two windows:

1. **Boresight Calibration Controls** (Tkinter) - Main control panel.
2. **Boresight Calibration Viewer** (Open3D) - 3D visualization window.

Boresight Calibration Workflow

1. Click "Point Cloud (.csv)" → select the downsampled file obtained from Script C (chapter 5).
2. Click "Trajectory (.txt)" → select the Pulsalyzer format trajectory file obtained from Script B (chapter 4).
3. Click "Apply & View" → loads files and shows initial point cloud.
4. Use the following mouse controls to navigate the 3D viewer:
    - **Left mouse button + drag** — rotate the view
    - **Middle mouse button + drag (or Ctrl + left click and drag)** — pan the view
    - **Scroll wheel** — zoom in/out

5. "Reset Camera View" button can be used to reset the camera view and centre the point cloud.
6. Adjust Roll (X), Pitch (Y), and Yaw (Z) sliders (Text inputs can also be used to set the values).
7. Note final values for use in the Python workflow (chapter 6).

8. F_pulsalyzer_real_time_visualizer.py

This script sends rotation commands to Pulsalyzer using the `A_json_commander.py` script, thus it can be used to visualize the effect of rotation on the Pulsalyzer-rendered point cloud in real-time.

8.1 How It Works

Pulsalyzer accepts a scanner transformation command (`set_scanner_transformation_deg`) via its JSON WebServer. This script sends the transformation command along with `reprocess_pointcloud` and `redraw_pointcloud` commands through `A_json_commander.py`, incrementally updating rotation angle along one axis and triggering a reprocess and redraw of the point cloud in Pulsalyzer.

The commands are sent through a `send_silent()` function which suppresses terminal output from `A_json_commander.py`, keeping the terminal clean during the loop.

8.2 User-Configurable Parameters

Variable	Default	Description
routing1	[...]	Routing path to the Pulsalyzer endpoint; must include the correct hostname
x_rot, y_rot, z_rot	0, 0, 0	Initial rotation angles (degrees)
start, stop	0, 10	Start and end values of the angle increment range

8.3 Prerequisites

- JSONServer and Pulsalyzer must be running, with Pulsalyzer having the required `.lidar` file and its corresponding trajectory loaded and processed.
- `A_json_commander.py` script must be in the same directory as this script.
- The hostname must match the device name visible in the Pulsalyzer WebServer interface (or `hostname` command in command prompt).

8.4 Usage

The script applies the initial rotation angles, pauses for a keypress, then enters an infinite loop cycling through the angle increment range. The loop restarts when the range is exhausted. Press `Ctrl+C` to stop the loop.

By default, the increment is applied to the Z-axis rotation, modify as needed for other axes. The update rate of point cloud in the 3D viewer of Pulsalyzer depends on the `point_step` parameter of commands `reprocess_pointcloud` and `redraw_pointcloud`.

This script was used in the thesis to identify the hidden rotation angle applied internally by Pulsalyzer. By iterating through a range of rotation angles ($-180°$ to $180°$) about the X and Z axes and observing the rendered point cloud in real-time, the angles at which the Pulsalyzer output visually matched the Python translation-only reference output was identified as $180°$, $0°$, $90°$ for X, Y, and Z axes respectively. This hidden rotation matrix was then used to derive the mathematical relationship between the Python and Pulsalyzer boresight angles.

9. G_pulsalyzer_angles_from_python.py

This script calculates the equivalent rotation angles for Pulsalyzer from the boresight calibration angles determined using the Python workflow. This conversion is necessary because Pulsalyzer has a hidden internal rotation and Python uses the **Intrinsic XYZ** rotation convention while Pulsalyzer uses **Extrinsic XYZ**.

9.1 Background

The two rotation conventions produce different rotation matrices for the same set of angle values. Additionally, Pulsalyzer applies an internal hidden pre-rotation of $[180°, 0°, 90°]$ around extrinsic X, Y, Z axes before applying the user-provided transformation angles.

9.2 Math

Let:

- R_{Python} = rotation matrix from Python intrinsic XYZ angles
- R_{internal} = Pulsalyzer's hidden pre-rotation: `Rotation.from_euler('xyz', [180, 0, 90], degrees=True)`
- R_{user} = rotation matrix that the user needs to pass to Pulsalyzer

Then:

$$R_{\text{user}} = R_{\text{internal}}^{T} \cdot R_{\text{Python}} \tag{9.1}$$

The user angles for Pulsalyzer are extracted from R_{user} using extrinsic XYZ decomposition.

9.3 User-Configurable Parameters

Variable	Default	Description
X, Y, Z	2.5, 184, 48	Boresight angles in degrees from Python workflow (Script D/E)

9.4 Running the Script

```
python G-pulsalyzer_angles_from_python.py
```

9.5 Output

For rotation angles of 2.5, 184, 48 in Python, Pulsalyzer angles: `rx=<value>, ry=<value>, rz=<value>`. Pass these values to Pulsalyzer using the `set_scanner_transformation_deg` command.

10. End-to-End Workflow Guide

10.1 Workflow 1 — Python Motion Compensation

Follow these steps in order:

1. **Export .las from Pulsalyzer** (without motion correction — set all transformation angles to 0).
2. **Run Script B** to convert the raw trajectory:
   - Set Year, Month, Day to match the scan date.
   - Set `prism_to_scanner_lever_arm` to the actual measured distance.
   - Run and verify output .txt exists.
3. **Run Script C** to generate downsampled CSVs for boresight calibration.
4. **Run Script E** to determine boresight angles:
   - Adjust sliders until the point cloud is geometrically correct.
   - Note the Roll, Pitch, Yaw values.
5. **Run Script D** to apply final motion compensation:
   - Set angles to the values from Step 4.
   - Run and verify the output `python_transformed_*.las` file.
6. **Open in CloudCompare** to inspect the result.

10.2 Workflow 2 — Pulsalyzer Motion Compensation

1. **Run Script B** (same as Python Workflow Step 2).
2. **Load .lidar file in Pulsalyzer** using identity quaternion configuration.
3. **Import trajectory** into Pulsalyzer using the converted .txt from Script B.
4. **Run Script G** to convert Python boresight angles to Pulsalyzer-equivalent angles.
5. **Apply angles in Pulsalyzer** using the command via Script F or the GUI.
6. **Export the result** as .las for comparison.

11. Data File Reference

11.1 Raw Trajectory File (input to Script B)

Space-separated .txt with no header:

```
HHMMSS.SSS X_old Y_old Z_old azimuth_gon zenith_gon slope_dist_m
101533.231 548775.866 5937527.313 10.950 208.58550 99.53554 35.764
```

- **Columns 2–4** (X_old, Y_old, Z_old) are not used.
- **Columns 5–7** (angles + distance) are what the script actually uses.
- **Timestamps** are UTC time-of-day only.

11.2 Pulsalyzer Trajectory File (output of Script B)

Tab-delimited .txt with header:

```
time x y z roll pitch yaw
1754438400.000000 -35.123456 0.012345 -0.234567 0.000000 0.000000
```

- **time:** Absolute UNIX-like timestamp in seconds (UTC + TAI offset).
- **x, y, z:** Cartesian coordinates in the total station frame (metres).
- **roll, pitch, yaw:** Scanner orientation — always 0 in this workflow.

11.3 Downsampled CSV (output of Script C, input to Script E)

Comma-separated .csv with header:

```
gps_time,x,y,z
581310.123456,0.12345678,0.23456789,-0.34567890
```

- **gps_time:** GPS Week Second (from the .las file).
- **x, y, z:** Point coordinates in the scanner's local coordinate frame.

12. Glossary of Key Terms

Term	Definition
ULi	Underwater LiDAR system developed by Fraunhofer IPM
Pulsalyzer	Proprietary software for the ULi system
Boresight calibration	Determining the relative orientation (rotation angles) between two sensors
Intrinsic XYZ	Rotation convention where each rotation is applied around the axes of the rotating frame (used in Python)
Extrinsic XYZ	Rotation convention where each rotation is applied around fixed world axes (used in Pulsalyzer) — equivalent to intrinsic ZYX
TAI-UTC offset	Difference between Atomic Time (TAI) and UTC (37 seconds as of 2025)
Lever arm	The vector offset between the prism (tracking point) and the ULi scanner lens
Motion compensation	Process of correcting point cloud distortion caused by scanner movement during data acquisition
GPS time	Timestamp stored in .las files — GPS Week Seconds (seconds since GPS epoch)
Rigid body trans.	A transformation consisting of rotation and translation only (no scaling or shearing)
R^2 value	Coefficient of determination used to verify that the trajectory is linear
Chunk processing	Loading and processing the .las file in fixed-size batches to manage memory usage
Gradians	Unit of angle measurement where 400 gradians = 360 degrees